



**What Limits Virtual Agent Application?
OmniBench: A Scalable Multi-Dimensional Benchmark for
Essential Virtual Agent Capabilities**

Research proposal

Автор: Бовкун Тимофей Дмитриевич

Материалы: https://github.com/bovtim/research_proposal_omnibench.git

Оригинальная статья: <https://openreview.net/pdf?id=4tFSK0Y2mT>

Материалы оригинального проекта: <https://omni-bench.github.io/>

Введение

Развитие MLLMs привело к скачку в разработке виртуальных агентов, способных взаимодействовать с GUI. Однако прогресс сдерживается несовершенством методов оценки. Традиционные бенчмарки обладают рядом ограничений. Во-первых, они характеризуются фиксированной и неконтролируемой сложностью задач. Во-вторых, создание наборов данных требует ручного аннотирования. В-третьих, большинство существующих подходов опираются либо на проверку конечного результата, либо на строгое сравнение траектории агента с эталонной демонстрацией человека, игнорируя промежуточный прогресс и вариативность путей решения.

Для решения данных проблем в рассматриваемой статье предложен графовый бенчмарк OmniBench и система оценки OmniEval.

Методология

Структура

Текстовые инструкции представляются в виде DAG. Узлы — множество атомарных подзадач, ребра — множество отношений зависимости между ними (зависимость возникает, если выходные ресурсы одной подзадачи выступают в качестве входных ресурсов другой).

Разработана интерактивная среда, включающая 49 приложений. Получение данных интерфейса обеспечивается скриншотами и структурным деревьям доступности.

Генерация тестовых данных

- Поиск подзадач.** MLLMs исследуют виртуальную среду, анализируют документацию к API и генерируют выполнимые инструкции для атомарных действий.
- Итеративный синтез.** MLLMs используются для выполнения сгенерированных инструкций. Параллельно кодовые модели генерируют функции для оценки выполнения подзадачи на базе 11 системных API. Алгоритм кросс-верификации итеративно оптимизирует и траектории, и оценочные функции.
- Построение графа задачи.** Система собирает подзадачи в единый граф, извлекая логическое «намерение» и предотвращая формирование бессмысленных последовательностей.
- Валидация.** Модель GPT-4o генерирует текстовую инструкцию, описывающую граф, а затем проверяет консистентность, пытаясь восстановить граф по тексту.

Оценка сложности задач

Измерение сложности	Топологический эквивалент
Сложность зависимостей	Количество ребер графа
Инструктивная сложность	Количество узлов (подзадач)
Сложность предметных знаний	Количество уникальных категорий ПО
Иерархическая сложность	Максимальная глубина графа
Сложность ветвления	Максимальная ширина графа

Оценка действий агента

Для оценки разработана архитектура OmniEval, функционирующая как динамическая машина состояний. Узлы проверяются посредством вызова сгенерированных функций. Рассчитываются две метрики.

1. **Coverage Rate (CR)** вычисляет взвешенный прогресс выполнения задачи:

$$w(s_i) = \frac{d(s_i)}{\sum_{j=1}^n d(s_j)}, \quad CR = \frac{\sum_{i=1}^n w(s_i) \mathbb{I}(s_i)}{\sum_{i=1}^n w(s_i)}$$

где $d(s_j)$ — глубина подзадачи, $\mathbb{I}(s_i)$ — индикатор выполнения задачи.

2. **Logical Consistency (LC)** оценивает схожесть поведения агента с человеческой логикой:

$$LC = \frac{CS_{agent}}{CS_{max}}$$

где CS_{agent} — количество выполненных последовательно пар подзадач в одной программе, CS_{max} — теоретически максимальный CS_{agent} по всем топологическим сортировкам графа.

Сильные и слабые стороны работы

Сильные стороны:

1. Экономическая и процедурная масштабируемость. Замещение ручной аннотации итеративным конвейером синтеза с кросс-верификацией между визуальными и кодовыми LLM позволяет генерировать задачи с минимальными затратами, а так же делает бенчмарк устойчивым к изменениям в дизайне интерфейсов приложений.
2. Использование графового представления и описанных выше метрик решает основные проблемы оценки работы агентов. Проверка статуса выполнения подзадач через API-состояния программ, поощряет агентов к поиску оптимальных путей решения задач и приближает тестирование к условиям реального мира. Кроме того, детализированный 10-мерный профиль способностей предоставляет высокоточный диагностический инструмент.

Слабые стороны:

1. Использование исключительно модели GPT-4o на этапе валидации консистентности вносит алгоритмическую предвзятость. Данные будут соответствовать паттернам рассуждения этой сети, что может исказить объективность.
2. Применение предопределенных API подходит для бинарных системных состояний (открытие файла, нажатие на гиперссылку), однако оказывается неэффективным при оценке качественных результатов генеративных задач (оценка художественного фильтра, монтаж видео). Это ограничивает применимость бенчмарка в данных сценариях.
3. Зависимость от фиксированного окружения и отсутствие псевдослучайных прерываний (системные уведомления, обрывы связи) создают тепличные условия для тестирования.

Предложения по развитию и улучшению

- **Интеграция визуальных фундаментальных моделей.** Для решения проблемы оценки творческих задач необходимо отказаться от исключительно детерминированных скриптов. Вместо обращений к API система должна иметь возможность вызывать VQA-модель для сравнения итогового графического холста с ожидаемым текстовым описанием, используя диффузионные метрики (например, LPIPS).
- **Внедрение псевдослучайных прерываний.** Поскольку агенты должны функционировать в реальном мире, предлагается внедрить в систему возможность генерации псевдослучайных прерываний (всплывающие системные диалоги, случайные изменения верстки окон). Это позволит изолированно измерять Error Recovery.

- **Модернизация метрик.** Coverage Rate опирается исключительно на глубину узла. Имеет смысл при вычислении веса узла так же основываться на косинусном сходстве векторных представлений инструкций соседних подзадач. Приведу пример:

Семантическая дистанция между узлами s_i и s_j :

$$SD(s_i, s_j) = 1 - \frac{v(s_i) \cdot v(s_j)}{\|v(s_i)\| \|v(s_j)\|}$$

$$w'(s_i) = d(s_i) + \lambda \sum_{s_j \in \mathcal{P}(s_i)} SD(s_i, s_j)$$

где $d(s_i)$ — глубина узла в графе, $\mathcal{P}(s_i)$ — множество родительских соседних узлов, λ — коэффициент масштабирования.

Метрика:

$$NACR = \frac{\sum_{i=1}^n w'(s_i) \mathbb{I}(s_i)}{\sum_{i=1}^n w'(s_i)}$$

Данная метрика позволит оценить, насколько эффективно агент управляет контекстом при переходах между логически связанными, но семантически удаленными подзадачами.

- **Решение проблемы алгоритмической предвзятости.** Необходимо заменить валидатор на базе GPT-4o консенсусным ансамблем моделей.
- **Тестирование кросс-платформенных сценариев.** Ввиду того, что современные пользовательские сценарии охватывают несколько устройств, архитектуру ресурсов следует расширить для поддержки межплатформенного взаимодействия.

Эксперимент

Изучим гипотезу, связанную с алгоритмической предвзятостью. В связи с отсутствием доступа к используемым в статье моделям, проверим идейную составляющую предположения.

В соответствии с описанием реализации из статьи были сгенерированы текстовые инструкции по 50 графам с помощью моделей GPT OSS 120B, Qwen3 235B A22B и DeepSeek 3.2. По сгенерированным инструкциям графы восстанавливались с помощью тех же моделей.

В таблице приведены значения F1-меры, представляющей собой среднее гармоническое между точностью и полнотой восстановления графов.

Модель-генератор (граф → инструкция)	Модель-восстановитель (инструкция → граф)		
	GPT OSS 120B	Qwen3 235B A22B	DeepSeek 3.2
GPT OSS 120B	0.79	0.80	0.25
Qwen3 235B A22B	0.69	0.92	0.12
DeepSeek 3.2	0.61	0.61	0.43

Среднее значение F1 на диагонали — 0.71, вне диагонали — 0.51. Данный результат подтверждает, что использование какой-то одной модели для валидации действительно вносит алгоритмическую предвзятость.

PS: с подробным отчетом и процессом можно ознакомиться в прилагаемом репозитории.

Список литературы

- [1] Bu, W., Wu, Y., Yu, Q., Gao, M., Miao, B., Zhang, Z., Pan, K., Li, Y., Li, M., Ji, W. et al. What Limits Virtual Agent Application? OmniBench: A Scalable Multi-Dimensional Benchmark for Essential Virtual Agent Capabilities. *arXiv preprint arXiv:2506.08933*, 2025.